Decentralized Validator-Backed Auditable Robust Aggregation for Federated Learning Against Poisoning and Aggregator Tampering

Shiqi Huang[a,*]

[a] *Faculty of Computational Mathematics and Cybernetics, Lomonosov Moscow State University, Moscow, Russia*

Abstract

Federated learning enables collaborative training without centralizing private data, but malicious clients can poison model updates and centralized aggregation servers create an additional trust bottleneck. A hash chain maintained by the same server that computes anomaly scores and aggregation weights is insufficient against an insider aggregator, because the server can fabricate a self-consistent audit trail. This paper proposes a decentralized validator-backed auditable robust aggregation framework for federated learning under client poisoning and aggregator-side tampering. In each round, a publicly selected aggregator proposes a reputation-aware robust aggregation decision. Independent validators verify client commitments, aggregator authorization, score and weight consistency, hash linkage, and proposal integrity before a threshold-signed audit block is finalized. The aggregation rule scores each client update using magnitude deviation, directional consistency, and historical stability, combines the score with smoothed reputation, and performs dynamic weighted aggregation. Experiments on Fashion-MNIST and CIFAR-10 cover IID and Dirichlet Non-IID settings under sign-flip, adaptive-scaling, and backdoor attacks. The proposed method avoids the catastrophic poisoning failures observed for unprotected FedAvg, maintains low attack success rate (ASR), and achieves competitive clean accuracy against norm-filtering and trimmed-mean baselines. Protocol-level experiments further show 100% valid-block finalization, 100% invalid-proposal rejection, 0% invalid-proposal

*Corresponding author.

Email address: shiqihuang666@gmail.com (Shiqi Huang)

acceptance, and explicit Byzantine-validator and validator-dropout boundaries under threshold finality.

Keywords: federated learning, poisoning attack, robust aggregation, validator committee, auditable aggregation, Byzantine robustness

1. Introduction

Federated learning (FL) allows multiple clients to collaboratively train a global model while keeping raw data local (McMahan et al., 2017; Kairouz et al., 2021). This paradigm is attractive for privacy-sensitive scenarios such as mobile computing, healthcare, industrial Internet of Things, and edge intelligence, and has motivated both production-scale system designs and heterogeneity-aware optimization methods (Bonawitz et al., 2019; Li et al., 2020a; Karimireddy et al., 2020; Reddi et al., 2021). From an information-security perspective, however, FL replaces centralized data collection with a new trust boundary: the server must aggregate updates from clients that may be faulty, compromised, or strategically malicious.

This trust boundary enables poisoning attacks. Malicious clients can manipulate uploaded model updates to degrade global accuracy, trigger targeted misclassification, or bypass defenses that assume poisoned updates are simple outliers (Fang et al., 2020; Bagdasaryan et al., 2020; Baruch et al., 2019; Cao et al., 2021). Classical robust aggregation rules, including Krum, coordinate-wise Median, Trimmed Mean, Bulyan, and geometric median, were designed to tolerate Byzantine clients by filtering or reducing the effect of abnormal updates (Blanchard et al., 2017; Yin et al., 2018; El Mhamdi et al., 2018; Pillutla et al., 2022). In Non-IID settings, the problem becomes harder because honest client updates may naturally differ from one another. This makes one-shot anomaly filtering vulnerable to both adaptive evasion and false rejection of honest clients.

Robustness alone is not sufficient for security-sensitive FL. When a client update is rejected or assigned a near-zero aggregation weight, the system should preserve the evidence behind that decision so that operators can reconstruct the training process, diagnose attacks, and audit whether benign heterogeneous clients were unfairly suppressed. Blockchain-assisted and decentralized FL have been proposed to improve transparency, coordination, incentive compatibility, and accountability (Lu et al., 2020; Nguyen et al., 2021; Dong et al., 2024; Zhang et al., 2024; Ren et al., 2024; Awan et al., 2022;

Yuan et al., 2024; Jiang et al., 2023). Nevertheless, simply recording model hashes or training metrics on chain does not directly improve robustness, and a hash chain controlled by the same aggregation server does not protect against aggregator-side insider manipulation. A more useful role for the ledger layer is to make the aggregation decision itself independently verifiable before it becomes part of the audit record.

This requirement is practical rather than merely diagnostic. In multi-party FL deployments such as cross-institutional analytics, industrial Internet of Things, and collaborative edge intelligence, operators may need to investigate security incidents, justify client exclusion, and demonstrate that model-governance rules were applied consistently. A defense that only outputs a final aggregate cannot support this type of accountability.

To address these issues, this paper proposes a decentralized validator-backed auditable robust aggregation framework. The key idea is to evaluate client updates through multiple signals while separating aggregation proposal from audit finalization. A round-specific aggregator proposes anomaly scores, reputation changes, aggregation weights, and model hashes, but the proposal is finalized only after independent validators verify the evidence and produce threshold signatures. Instead of permanently accepting or rejecting clients using a static threshold, the framework assigns dynamic aggregation weights based on both current anomaly scores and historical trust. This design aims to improve robustness against adaptive poisoning while making aggregation decisions reconstructable and resistant to unilateral manipulation by a dishonest aggregator.

The main contributions are:

1. We formulate a multi-signal anomaly score for FL updates that combines norm deviation, directional consistency, and history stability under heterogeneous client data.
2. We design a validator-backed audit mechanism that records the evidence behind aggregation weights and prevents a single aggregator from unilaterally finalizing the audit chain.
3. We introduce a dynamic weighted aggregation rule that couples current anomaly evidence with historical reputation to reduce malicious influence while retaining useful benign Non-IID updates.
4. We provide a reproducible benchmark covering IID and Non-IID data, multiple poisoning attacks, three random seeds for the main Fashion-MNIST and CIFAR-10 Non-IID settings, malicious-ratio sensitivity, au-

dit reconstruction, invalid-proposal rejection, Byzantine-validator boundaries, validator-dropout liveness, and overhead measurement.

2. Related Work

2.1. Federated Learning Under Heterogeneity

FedAvg established a communication-efficient training paradigm for decentralized data (McMahan et al., 2017), and subsequent work has clarified that FL differs from conventional distributed optimization because clients are statistically and systemically heterogeneous (Kairouz et al., 2021). FedProx addresses client drift through a proximal regularizer (Li et al., 2020a), SCAFFOLD reduces drift through control variates (Karimireddy et al., 2020), and adaptive federated optimization improves server-side update dynamics (Reddi et al., 2021). These optimization methods are not poisoning defenses, but they motivate the experimental emphasis on Non-IID data: under heterogeneous client distributions, benign updates may naturally deviate from the global mean. A robust security mechanism therefore needs to distinguish malicious deviation from legitimate heterogeneity.

2.2. Poisoning and Backdoor Attacks

Poisoning attacks in FL can be categorized into data poisoning and model poisoning (National Institute of Standards and Technology, 2024; Mothukuri et al., 2021). Data poisoning manipulates local training data or labels, while model poisoning directly modifies the uploaded model update. Backdoor attacks aim to preserve clean accuracy while causing targeted misclassification when a trigger appears, and distributed backdoor attacks further exploit the multi-client structure of FL (Bagdasaryan et al., 2020; Xie et al., 2020; Nguyen et al., 2022). Attack studies on Byzantine-robust FL emphasize adaptive adversaries that understand the aggregation rule and shape poisoned updates to evade detection (Fang et al., 2020; Baruch et al., 2019; Wang et al., 2025b). Sybil-style poisoning also shows that persistent client behavior and update similarity can be useful security signals (Fung et al., 2020).

2.3. Robust Aggregation

Robust aggregation methods such as Krum, Median, Trimmed Mean, Bulyan, and geometric median reduce the impact of abnormal client updates (Blanchard et al., 2017; Yin et al., 2018; El Mhamdi et al., 2018; Pillutla

et al., 2022). Trust-based approaches such as FLTrust use a trusted reference update to compute client trust scores (Cao et al., 2021). Backdoor-oriented defenses such as FLAME consider clustering, clipping, and noise injection (Nguyen et al., 2022). Recent adaptive defenses further explore reinforcement-learning-based aggregation and history-aware heterogeneous FL recovery (Wang et al., 2025b,a). These methods motivate stronger experimental baselines and show that single-signal defenses are often insufficient.

2.4. Trust and Reputation Signals

Trust and reputation mechanisms are attractive in FL because poisoning behavior unfolds over rounds rather than in a single isolated upload. FLTrust uses a server-side root dataset to construct a trusted reference direction (Cao et al., 2021), while FoolsGold uses update similarity to reduce the influence of Sybil-style attackers (Fung et al., 2020). Blockchain-assisted FL systems also frequently incorporate incentives or punishment records to discourage low-quality or malicious updates (Lu et al., 2020; Zhang et al., 2024; Ren et al., 2024). However, trust signals must be handled carefully under Non-IID data: a benign client with rare classes may appear directionally unusual, while an adaptive attacker may keep update norms close to benign statistics. This motivates reputation updates that combine current anomaly evidence with historical behavior rather than relying on a single trust cue.

2.5. Blockchain-Assisted, Decentralized, and Validator-Backed Federated Learning

Blockchain can provide tamper-resistant logging, decentralized coordination, incentives, and accountability. Early decentralized and blockchain-assisted FL systems such as Biscotti, BLADE-FL, and BlockDFL show that ledger-based coordination can reduce dependence on a trusted central server (Shayan et al., 2018; Li et al., 2021; Qin et al., 2022). Surveys of decentralized and blockchained FL further identify topology, communication, privacy, data heterogeneity, trust management, and compatibility overhead as central challenges (Yuan et al., 2024; Jiang et al., 2023; Hallaji et al., 2024). Recent blockchain-FL defenses increasingly connect ledger records with poisoning detection, robust aggregation, validators, and consensus mechanisms (Dong et al., 2024; García-Márquez et al., 2025; Carney et al., 2025; Bouchiha et al., 2026).

Committee-based designs are particularly relevant. BFLC uses committee consensus to reduce consensus cost in blockchain-based FL, while VBFL

introduces validator-based local-model validation and proof-of-stake-inspired consensus (Li et al., 2020b; Chen et al., 2021). These designs show that validators should not only store blocks but also check model or update legitimacy. At the same time, BFT consensus literature makes clear that safety and liveness depend on explicit fault bounds, validator availability, and communication overhead (Zhang et al., 2022; Alqahtani and Demirbas, 2021). Committee selection also creates security questions, including Sybil resistance, validator collusion, and static-committee capture (Rajab et al., 2020). The present work focuses on this narrower but important point: making robust aggregation decisions independently verifiable before they are finalized into an audit chain.

2.6. Research Gap

Existing robust aggregation rules mainly optimize the current-round aggregate, while ledger-assisted FL often treats the ledger as a storage, incentive, or coordination layer. These two lines of work are complementary but not tightly coupled. A rejected or down-weighted update should be explainable after training: the system should record which anomaly signals were observed, how reputation changed, and how those signals affected the final aggregation weight. More importantly, if the aggregator itself may be dishonest, the evidence ledger cannot be finalized solely by that same aggregator. The gap addressed in this paper is therefore not a new standalone norm filter and not a full production blockchain deployment. The contribution is the coupling of multi-signal anomaly scoring, persistent reputation-aware weighting, client commitments, rotating aggregator proposals, validator verification, and threshold-finalized audit records so that robustness decisions are both operational during training and independently reconstructable afterward.

3. Threat Model and Problem Definition

We consider a permissioned consortium FL system with N clients, a set of eligible round aggregators, and a validator committee $\mathcal{V}$ of size m. At communication round t, the current global model parameters w_t are made available to selected clients. Client i trains locally on its private dataset D_i and returns a local model $w_{t,i}$ or, equivalently, an update vector

$$u_{t,i} = \mathrm{vec}(w_{t,i} - w_t), \tag{1}$$

where vec$(\cdot)$ denotes concatenation of all floating-point model parameters. Client i has $n_i = |D_i|$ training samples. A subset $\mathcal{M}_t$ of clients may be malicious. Malicious clients can poison uploaded model updates, collude with each other, and adapt update magnitudes to evade simple norm-based thresholds.

The aggregation role is not assumed to be permanently trusted. A public leader-selection rule chooses a round aggregator for each round. The selected aggregator proposes anomaly scores, reputation updates, aggregation weights, model hashes, and an audit payload. A dishonest aggregator may alter scores, change aggregation weights, omit clients, introduce fake clients, modify model hashes, equivocate across proposals, or attempt to finalize a self-serving audit record. The aggregator can propose a block but cannot finalize it alone.

Validators are permissioned entities that verify proposed audit blocks. A block is finalized only if at least q validators sign it. We consider Byzantine validators that may sign invalid proposals and offline validators that do not respond. The protocol's safety is meaningful when fewer than q validators sign an invalid proposal; if q Byzantine or lazy validators collude to sign an invalid proposal, threshold finality can be subverted, although the signatures provide accountability evidence. Liveness requires at least q available validators. This work therefore claims threshold-finalized auditability under explicit committee assumptions, not unconditional security against validator-majority collusion or permissionless Sybil attacks.

The defense objective is to compute aggregation weights $a_{t,i}$ such that malicious influence is reduced while benign updates remain useful under Non-IID data. The aggregated model is

$$\beta_{t,i} = \frac{a_{t,i} n_i}{\sum_j a_{t,j} n_j}, \qquad w_{t+1} = \sum_i \beta_{t,i} w_{t,i}. \tag{2}$$

When $a_{t,i} = 0$, client i has no influence on the round. When all weights become zero, the implementation falls back to the lowest-anomaly fraction of clients to avoid a stalled round. In addition to utility and robustness, the system must preserve an auditable record of the evidence and decision that produced each $a_{t,i}$, and that record must be verifiable by parties other than the aggregator that proposed it.

4. Proposed Method

4.1. Overview

Each communication round contains five steps. First, clients train locally and submit signed commitments to their updates. Second, a public rule selects the round aggregator from the eligible aggregator set. Third, the selected aggregator computes anomaly signals, reputation updates, and aggregation weights, then proposes an audit block containing the decision evidence. Fourth, validators independently verify client commitments, aggregator authorization, score and weight consistency, hash linkage, and proposal integrity. Finally, the audit block is finalized only if at least q validators sign it. This separates decision proposal from audit finalization: the aggregator can compute and propose the robust aggregation decision, but the validator committee controls whether the decision becomes part of the finalized audit chain.

4.2. Multi-Signal Anomaly Score

For each round, the selected aggregator computes the update norm

$$r_{t,i} = \|u_{t,i}\|_2. \tag{3}$$

The robust norm center is the median update norm:

$$c_t = \mathrm{median}_i(r_{t,i}). \tag{4}$$

The robust scale is the median absolute deviation (MAD):

$$s_t = \max\{\mathrm{median}_i(|r_{t,i} - c_t|), \epsilon\}, \tag{5}$$

where ϵ is a small constant used to avoid division by zero. The one-sided norm anomaly score is

$$z^n_{t,i} = \max\left(0, \frac{r_{t,i} - c_t}{s_t}\right). \tag{6}$$

The reference update direction is the mean update

$$\bar{u}_t = \frac{1}{N}\sum_i u_{t,i}. \tag{7}$$

The directional consistency score uses cosine similarity:

$$d_{t,i} = \cos(u_{t,i}, \bar{u}_t), \qquad z^d_{t,i} = \max(0, 1 - d_{t,i}). \tag{8}$$

If client i participated in the previous round with stored update $u_{t-1,i}$, the history instability score is

$$z^h_{t,i} = \max(0, 1 - \cos(u_{t,i}, u_{t-1,i})). \tag{9}$$

For a new client without history, $z^h_{t,i} = 0$. The final anomaly score is

$$A_{t,i} = 0.45 z^n_{t,i} + 0.35 z^d_{t,i} + 0.20 z^h_{t,i}. \tag{10}$$

4.3. Reputation and Client Commitments

Each client has a reputation value $R_{t,i}$ initialized as $R_{0,i} = 1$. After computing anomaly score $A_{t,i}$, reputation is updated by exponential smoothing:

$$R_{t+1,i} = \text{clip}\left(0.85 R_{t,i} + 0.15 \exp(-A_{t,i}), 0.05, 1.5\right). \tag{11}$$

The clipping interval prevents permanent reputation collapse. It also bounds the maximum advantage of a client with consistently benign behavior.

Each client also signs a lightweight commitment containing round ID, client ID, update hash, sample count, and metadata hash. The commitment is not intended to reveal raw data or store full model updates on chain. Its purpose is to prevent a dishonest aggregator from inventing a client update, silently changing client metadata, or omitting submitted evidence without leaving a detectable mismatch between client commitments and the proposed audit payload.

4.4. Aggregator Proposal and Validator Finality

Let P_t denote the round payload containing model hash, metrics, client metadata, anomaly scores, reputation values, aggregation weights, and client commitments. The eligible aggregator for round t is selected by a public deterministic rule in the current implementation:

$$g_t = \text{sorted}(\mathcal{G})[(t-1) \bmod |\mathcal{G}|], \tag{12}$$

where $\mathcal{G}$ is the permissioned aggregator set. The round-robin rule is used for reproducibility in the simulator. It can be replaced in deployment by a BFT

leader-rotation rule, verifiable randomness, or a reputation-aware selection mechanism.

The selected aggregator signs a proposal containing the round number, aggregator ID, previous block hash, payload hash, and proposal hash. Validators then check: (i) the previous hash matches the latest finalized block; (ii) the payload hash and proposal hash are correct; (iii) the aggregator is eligible for the round and its signature is valid; (iv) client commitments match the client records and signatures; (v) the client, decision, and commitment sets are consistent; and (vi) rejection flags and aggregation weights are internally consistent with the recorded decision evidence.

Each finalized audit block is

$$B_t = (t, \text{timestamp}_t, h_{t-1}, P_t), \qquad h_t = \text{SHA256}(\text{canonical\_json}(B_t)). \quad (13)$$

The next block stores h_t as its previous hash. Verification checks both hash correctness and hash linkage from the genesis hash to the final block. In the validator-backed design, B_t is finalized only after at least q validators accept and sign the proposal.

The implemented audit layer is a permissioned threshold-finality simulation that models the audit component of a blockchain-assisted FL system. It does not implement full PBFT/HotStuff/Tendermint message flow, view change, networking, validator incentives, smart-contract execution, or gas accounting. This scope isolates the core scientific question of whether robust aggregation decisions can be made reconstructable and independently verifiable under client poisoning and aggregator-side tampering.

4.5. Dynamic Weighted Aggregation

The selected aggregator first applies hard-rejection rules. Client i is hard rejected if any of the following conditions holds:

$$A_{t,i} > \tau_A, \qquad z^n_{t,i} > \tau_n \text{ and } d_{t,i} < 0, \qquad d_{t,i} < \tau_d. \quad (14)$$

The implemented default parameters are $\tau_A = 4.0$, $\tau_n = 2.5$, and $\tau_d = -0.4$. If a client is not hard rejected, its raw aggregation weight is

$$a_{t,i} = \max(a_{\min}, R_{t+1,i} \exp(-\gamma A_{t,i})), \quad (15)$$

where $a_{\min} = 0$ and $\gamma = 1.2$ in the formal experiments. If the hard-rejection condition is true, $a_{t,i} = 0$. The final normalized aggregation coefficient is sample-size weighted according to Eq. (2). If all raw weights are zero, the implementation selects the lowest-anomaly 50% of clients as a fallback set and assigns them exponentially decayed weights.

4.6. Security and Audit Interpretation

The proposed framework provides validator-backed decision traceability rather than a cryptographic proof that every malicious client is detected. For each round, the audit payload records the evidence used by the aggregation rule: update hashes, anomaly scores, reputation values before and after smoothing, hard-rejection flags, raw aggregation weights, normalized aggregation coefficients, and global evaluation metrics. Because the block hash is computed over a canonical representation of this payload and linked to the previous block hash, any post-hoc modification of a past decision changes the corresponding block hash and breaks the subsequent chain linkage. Because the proposal is finalized only with validator signatures, a single dishonest aggregator cannot unilaterally rewrite or finalize its own audit trail.

The audit record supports four verification tasks. First, an auditor can recompute the hash chain to check whether the recorded training history has been modified. Second, given the stored round payload, the auditor can reconstruct why a client received zero, reduced, or full aggregation influence. Third, validators can reject invalid proposals from unauthorized or dishonest aggregators before finality. Fourth, an auditor can inspect validator signatures and rejected votes to identify which validators approved or rejected a proposal. These checks are useful for accountability and forensic analysis, but they do not replace secure aggregation, differential privacy, trusted execution, full blockchain consensus, or permissionless identity management. Those mechanisms address different security goals and can be layered with the proposed aggregation logic in future deployments.

4.7. Decision Semantics and Audit Evaluation Protocol

The protocol follows three lines of prior work: Byzantine-robust aggregation evaluates whether malicious updates lose influence in the aggregate (Blanchard et al., 2017; Yin et al., 2018; Pillutla et al., 2022), trust-aware FL uses client-level evidence to weight or suppress suspicious participants (Cao et al., 2021; Fung et al., 2020), and blockchain-assisted FL emphasizes tamper-resistant records and verifiable training histories (Lu et al., 2020; Nguyen et al., 2021; Dong et al., 2024; Awan et al., 2022). Our audit protocol adapts these ideas to client-round decision tracing: it evaluates not only the final model utility but also whether the evidence behind each aggregation decision can be reproduced after training.

To make the audit interpretation explicit, each client-round decision is mapped to one of four states. A client is *accepted* when it does not trigger

a hard-rejection rule and receives a nonzero aggregation weight close to the benign-client range in that round. A client is *down-weighted* when it does not trigger hard rejection but receives a reduced aggregation weight because its anomaly score increases or its reputation decreases. A client is *rejected* when a hard-rejection rule sets $a_{t,i} = 0$, meaning the client has no influence on the aggregate in that round. Finally, a client is *fallback selected* only when all raw weights are zero and the lowest-anomaly clients are restored with small positive weights to avoid a stalled training round.

The explanation attached to a decision is not a free-form textual justification; it is a reconstructable record derived from the stored evidence. For each client and round, the audit payload stores the norm anomaly score, direction score, history score, combined anomaly score, reputation before and after updating, hard-rejection flag, raw aggregation weight, normalized aggregation coefficient, and fallback flag if applicable. Given these fields and the fixed thresholds in Table 1, an auditor can reproduce whether the client was accepted, down-weighted, rejected, or fallback selected, and can identify which signal primarily caused the decision.

We evaluate auditability through four protocol-level checks. First, *chain validity* verifies that all block hashes and previous-hash links are intact. Second, *decision reconstructability* checks whether the stored anomaly, reputation, threshold, and weight fields are sufficient to reproduce the recorded aggregation decision. Third, *malicious influence suppression* is measured in controlled experiments by the mean aggregation weight and zero-weight rate assigned to known malicious clients. Fourth, *benign retention* is inspected through nonzero aggregation weights for benign clients under Non-IID data. These metrics evaluate the traceability of aggregation decisions; they should not be interpreted as a guarantee that every malicious client is perfectly detected in deployment.

4.8. Algorithm

Algorithm 1: Validator-backed auditable robust aggregation. Given the current global model w_t, client models $\{w_{t,i}\}$, sample counts $\{n_i\}$, reputation table $\{R_{t,i}\}$, previous updates $\{u_{t-1,i}\}$, eligible aggregators $\mathcal{G}$, validator committee $\mathcal{V}$, threshold q, and previous finalized hash h_{t-1}:

1. Each selected client submits a signed commitment over round ID, client ID, update hash, sample count, and metadata hash.
2. Select the round aggregator g_t using the public aggregator-selection rule.

Table 1: Proposed aggregation hyperparameters.

Parameter	Value	Meaning
Norm coefficient	0.45	Norm-deviation weight in $A_{t,i}$
Direction coefficient	0.35	Direction-inconsistency weight in $A_{t,i}$
History coefficient	0.20	History-instability weight in $A_{t,i}$
τ_A	4.0	Anomaly hard-rejection threshold
τ_n	2.5	Norm threshold used with negative direction
τ_d	-0.4	Direction threshold for hard rejection
γ	1.2	Exponential weight temperature
$a_{\min}$	0.0	Minimum aggregation weight before fallback
Reputation clip	[0.05, 1.5]	Lower and upper reputation bounds
Fallback fraction	0.5	Lowest-anomaly fallback fraction

3. The aggregator computes update vectors $u_{t,i}$, norms $r_{t,i}$, median norm center c_t, MAD scale s_t, and mean update $\bar{u}_t$.
4. The aggregator computes norm, direction, and history anomaly scores for each client.
5. The aggregator updates reputation values $R_{t+1,i}$, computes hard-rejection decisions and aggregation weights $a_{t,i}$, and applies fallback selection if all raw weights are zero.
6. The aggregator computes w_{t+1} using sample-size-normalized weights and proposes an audit block containing hashes, metrics, anomaly evidence, reputation values, aggregation weights, rejection decisions, client commitments, and aggregator signature.
7. Each validator verifies client commitments, aggregator eligibility, proposal signature, hash linkage, required decision fields, and aggregation-weight consistency.
8. If at least q validators accept and sign the proposal, finalize the audit block and set h_t to the finalized block hash; otherwise the block remains unfinalized and the round requires recovery.

The output is the updated global model, reputation table, previous-update cache, validator-signed audit block, and finalized audit hash.

5. Experimental Setup

5.1. Datasets and Models

MNIST is used for sanity checks, while Fashion-MNIST and CIFAR-10 serve as the main benchmarks. Fashion-MNIST is evaluated under both IID and Dirichlet Non-IID settings. CIFAR-10 is used to test whether the conclusions remain valid on a harder colored-image benchmark, with emphasis on the more challenging Non-IID setting. Because CIFAR-10 is substantially more expensive in the current FL simulator, we use it primarily as a three-seed Non-IID stress test rather than exhaustively repeating every IID setting.

5.2. Data Distribution, Attacks, and Baselines

The main Non-IID experiments use a Dirichlet split with $\alpha = 0.5$. Each main setting uses 10 clients with a malicious ratio of 0.2 unless otherwise specified. The main attack set includes sign flipping, adaptive scaling, and backdoor trigger attacks. The implemented empirical baselines are FedAvg, Krum, Trimmed Mean, coordinate-wise Median, and norm filtering. These baselines cover unprotected averaging, distance-based selection, coordinate-wise robust statistics, and a strong norm-based filter.

FLTrust and FLAME are discussed as closely related defenses but are not included in the current empirical tables. FLTrust relies on a trusted server-side root dataset to construct a reference update, which changes the trust assumptions of the benchmark. FLAME is designed specifically for backdoor mitigation through clustering, clipping, and noise injection. Faithful implementations of these two methods would strengthen a broader comparison, but the current study focuses on whether audit evidence, reputation history, and dynamic robust aggregation can be coupled without assuming trusted root data or a backdoor-specific defense pipeline.

5.3. Metrics and Reproducibility

The evaluation metrics include clean accuracy, best clean accuracy, test loss, attack success rate (ASR), rejection or zero-weight rate, chain validity, and runtime. Rejection-based metrics should be interpreted as aggregation decisions rather than perfect malicious-client detection labels. For auditability, we further report objective metrics: chain verification rate, simulated tamper-detection rate, decision reconstruction rate, malicious suppression rate, benign retention rate, and benign–malicious aggregation-weight

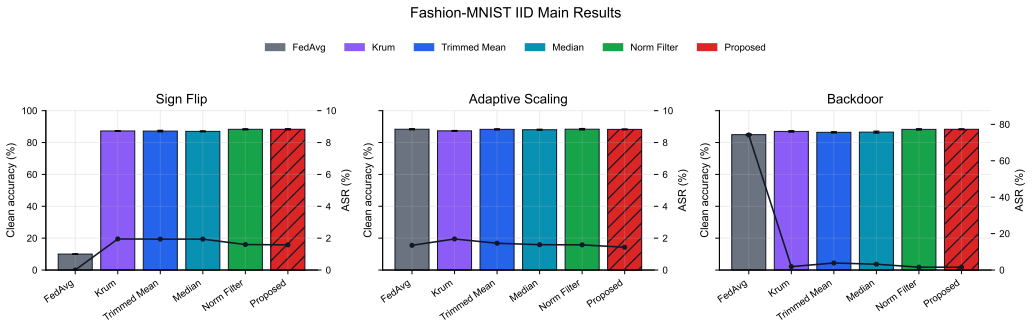

Figure 1: Clean accuracy and attack success rate on Fashion-MNIST under IID data. Bars report mean clean accuracy over three seeds, error bars report standard deviation, and the line reports ASR.

gap. For validator-backed finality, we report valid-block finalization rate, aggregator-authorization verification rate, invalid-proposal rejection rate, invalid-proposal acceptance rate, Byzantine-boundary behavior, validator-dropout liveness, and verification latency. These metrics are computed from the recorded audit logs and protocol simulations rather than manually assigned after training.

All formal result tables are generated from the consolidated summary CSV using deterministic table-export scripts. The scripts keep the latest run for each experiment name and exclude smoke tests, tuning diagnostics, and extended exploratory attacks. Figures and audit-case materials are generated by the corresponding export scripts in the `experiments/` directory.

6. Results and Discussion

6.1. Fashion-MNIST Main Results

On Fashion-MNIST, FedAvg collapses under sign flipping, reaching approximately random accuracy in both IID and Non-IID settings. FedAvg also remains vulnerable to backdoor attacks: although clean accuracy remains around 85%, ASR is high, especially under Non-IID backdoor settings. Krum avoids some attacks but suffers substantial clean-accuracy degradation under Non-IID data, confirming that distance-based single-update selection is brittle under heterogeneous client distributions.

The proposed method is stable across all Fashion-MNIST main settings and does not exhibit the catastrophic divergence observed in unprotected FedAvg. Under IID settings, it is essentially tied with norm filtering: for sign

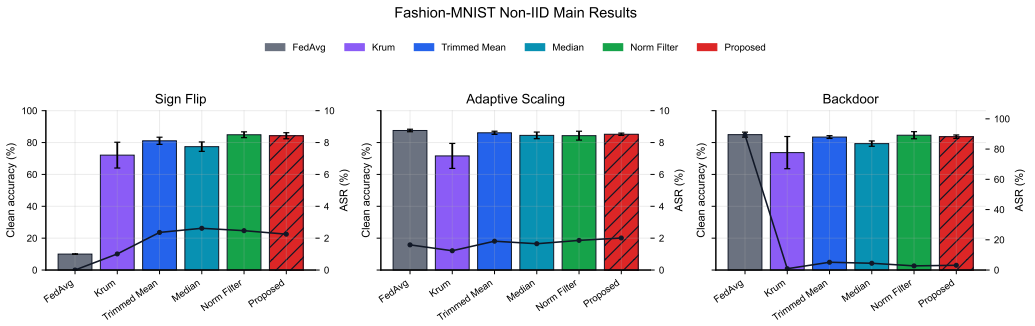

Figure 2: Clean accuracy and attack success rate on Fashion-MNIST under Dirichlet Non-IID data. FedAvg collapses under sign-flip attacks and shows high ASR under backdoor attacks, while robust aggregation methods maintain competitive clean accuracy and suppress ASR.

flipping, backdoor, and adaptive scaling, the proposed method reaches approximately 88.3% clean accuracy with ASR around 1.4–1.6%. Under Non-IID settings, it remains competitive, reaching 84.27% under sign flipping, 85.21% under adaptive scaling, and 83.67% under backdoor attacks. However, the method is not uniformly the top clean-accuracy method; Trimmed Mean and norm filtering remain strong competitors in some Fashion-MNIST settings.

These results illustrate two practical security points. First, unprotected averaging is not merely less accurate under attack; in sign-flip settings it can converge to a nearly unusable model, which is unacceptable for security-sensitive FL services. Second, Krum's poor Non-IID behavior shows that selecting a single apparently central update can discard useful benign diversity. The proposed method avoids this failure mode by assigning continuous aggregation weights rather than selecting one update or applying only a binary norm threshold. Its advantage is therefore stability and traceability, not guaranteed dominance in clean accuracy.

6.2. CIFAR-10 Non-IID Results

CIFAR-10 Non-IID experiments provide a harder validation setting. FedAvg again collapses under sign flipping, while Krum shows large instability and low clean accuracy across attacks. Norm filtering is the strongest clean-accuracy baseline on CIFAR-10 Non-IID, reaching 77.62% under sign flipping, 81.45% under adaptive scaling, and 77.88% under backdoor attacks.

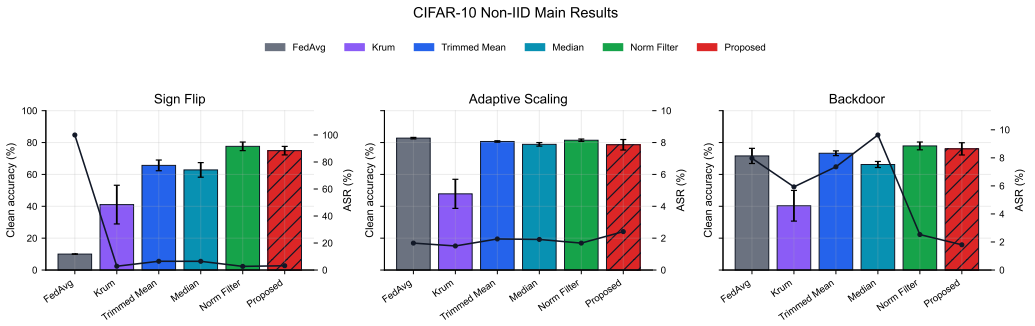

Figure 3: Clean accuracy and attack success rate on CIFAR-10 under Dirichlet Non-IID data. Norm filtering achieves the highest clean accuracy in most settings, while the proposed method remains competitive and maintains low ASR with auditable aggregation decisions.

The proposed method obtains 74.91%, 78.61%, and 76.04% clean accuracy under sign flipping, adaptive scaling, and backdoor attacks, respectively. Its ASR remains low, especially under backdoor attacks, but clean accuracy trails norm filtering by roughly 1.8–2.8 percentage points. Therefore, the correct empirical claim is not that the proposed method is universally more accurate than all baselines. Instead, the evidence supports a more precise claim: the proposed method provides auditable, reputation-aware robustness with competitive clean accuracy and low ASR, while norm filtering is a stronger pure clean-accuracy baseline on CIFAR-10 Non-IID.

The CIFAR-10 results are important because they prevent overstating the proposed method. A simple norm-filtering rule is highly effective on this benchmark. This suggests that update magnitude remains a strong signal under the evaluated attacks. However, norm filtering alone does not record why each client was down-weighted, how the decision evolved over rounds, or how the decision can be reconstructed after training. The proposed method pays a modest clean-accuracy cost in this harder setting while adding reputation history and tamper-evident decision records. This trade-off is acceptable when post-hoc accountability is part of the security requirement, but it would not justify replacing norm filtering in a deployment whose only objective is maximum clean accuracy.

6.3. Malicious-Ratio Sensitivity

The malicious-ratio sensitivity experiment uses Fashion-MNIST Non-IID adaptive scaling with malicious ratios of 10%, 20%, and 40%. Proposed,

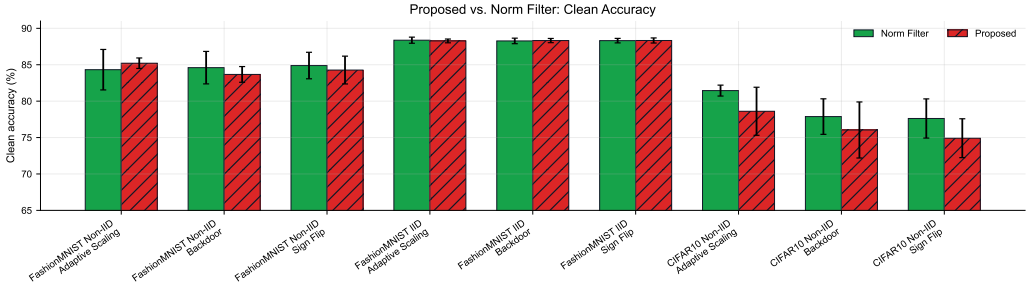

Figure 4: Clean-accuracy comparison between the proposed method and norm filtering across main benchmark conditions.

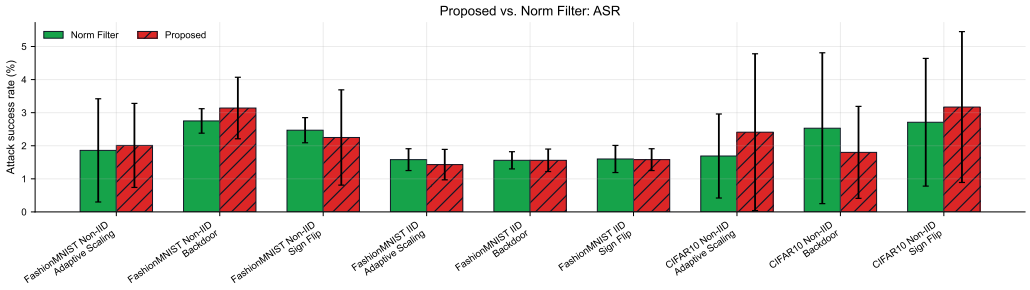

Figure 5: Attack-success-rate comparison between the proposed method and norm filtering across main benchmark conditions.

Trimmed Mean, and norm filtering all remain stable with no early stopping. The proposed method maintains approximately 85.0–85.2% clean accuracy and about 2.0% ASR across all three ratios. Trimmed Mean has the best average clean accuracy in this specific sensitivity setting, while norm filtering improves as the malicious ratio increases. These results indicate stability under increasing malicious participation, but they do not support a claim of universal superiority for the proposed method.

The absence of early stopping across these ratios suggests that the evaluated robust methods have a useful safety margin against adaptive-scaling attacks in this setting. At the same time, the small differences among robust methods indicate that malicious-client ratio alone is not enough to characterize defense difficulty. Attack objective, data heterogeneity, model architecture, and the defense's decision rule jointly determine whether benign and malicious updates are separable.

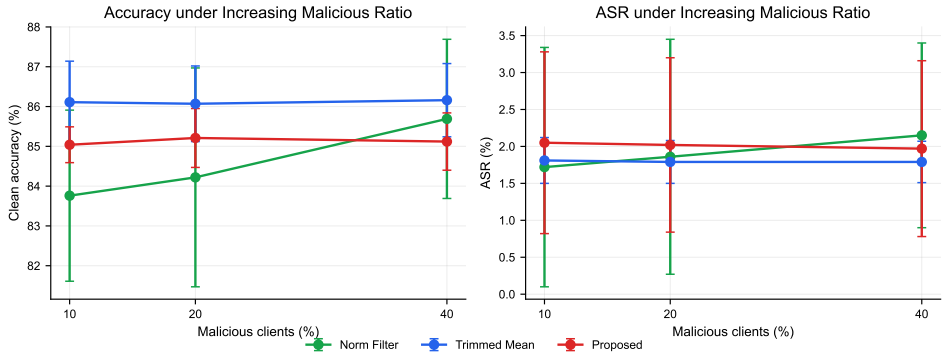

Figure 6: Sensitivity to malicious-client ratio on Fashion-MNIST Non-IID adaptive-scaling attacks. All three robust methods remain stable from 10% to 40% malicious clients, with no early stopping.

6.4. Auditability

For every completed run, the framework writes per-round metrics, per-client aggregation decisions, and hash-linked audit records. This audit trail is the main system-level distinction from ordinary robust aggregation baselines. The paper emphasizes auditability and reproducible decision tracing as first-class design goals, while clean accuracy and ASR demonstrate that the audit mechanism is paired with a practically competitive defense.

A representative audit case uses Fashion-MNIST Non-IID sign-flip attack with the proposed method and seed 42. The run contains 80 audit blocks and passes hash-linkage verification, satisfying the chain-validity check. The stored per-client fields also support decision reconstruction because each zero-weight or reduced-weight decision can be traced to the recorded anomaly scores, reputation values, rejection flags, and aggregation weights. The final clean accuracy is 86.47%, the best clean accuracy is 86.47%, and the final ASR is 1.08%. The two malicious clients receive zero mean aggregation weight across all 80 rounds, while benign clients retain an average aggregation weight of 0.2831. This case demonstrates malicious influence suppression and benign retention in the controlled benchmark, while also showing how the recorded evidence supports post-hoc inspection of aggregation decisions.

Table 2 quantifies this auditability claim. Chain verification succeeds for the recorded audit chain, and a simulated tamper test detects all single-block modifications of recorded metrics. The decision reconstruction rate is 100%, meaning that all 800 client-round aggregation decisions in the case

Table 2: Auditability and client-influence metrics for the representative audit case.

Metric	Value	Interpretation
Stored audit blocks	80	Number of hash-linked round records.
Chain verification rate	100.00%	Recorded hashes and previous-hash links are valid.
Tamper detection rate	100.00%	All simulated single-block metric modifications are detected.
Decision reconstruction rate	100.00%	All 800 client-round decisions are reproducible from stored decision fields.
Malicious suppression rate	100.00%	Malicious client-rounds receive zero aggregation weight.
Benign retention rate	85.78%	Benign client-rounds retain nonzero aggregation weight.
Benign–malicious weight gap	0.2831	Difference between mean benign and malicious aggregation weights.
Mean rejection precision / recall	64.38% / 100.00%	Zero-weight decisions suppress all malicious client-rounds but also conservatively reject some benign Non-IID updates.

can be reproduced from the stored decision fields. Malicious client-rounds receive zero aggregation weight in all cases, while 85.78% of benign client-rounds retain nonzero contribution. The mean rejection precision is lower than recall because several benign Non-IID clients are conservatively assigned zero weight in some rounds. This confirms that the rejection signal should be interpreted as a cautious aggregation decision rather than a perfect malicious-client detector.

To verify that these auditability properties are not limited to one selected case, we also compute the same objective audit metrics across all latest formal proposed-method runs after removing superseded duplicate runs. This multi-run audit evaluation covers the main benchmark settings and the malicious-ratio sensitivity study. As shown in Table 3, all 39 proposed-method runs pass chain verification, simulated tamper detection, and decision reconstruction. In total, the audit logs contain 3600 hash-linked blocks and 36000 client-round decisions. Benign retention remains high on average, but it varies across attacks and data heterogeneity because the defense sometimes assigns zero weight to benign Non-IID clients conservatively. This is why rejection-related metrics are reported as aggregation-decision statistics rather than as perfect detection metrics.

Table 3: Multi-run auditability summary for latest formal proposed-method runs. Values are mean ± standard deviation over runs.

Scope	Runs / blocks / decisions	Chain verification	Tamper detection	Decision reconstruction / benign retention
Main benchmark	30 / 2880 / 28800	100.00 ± 0.00%	100.00 ± 0.00%	100.00 ± 0.00% / 95.56 ± 6.48%
Malicious-ratio sensitivity	9 / 720 / 7200	100.00 ± 0.00%	100.00 ± 0.00%	100.00 ± 0.00% / 100.00 ± 0.00%
All proposed runs	39 / 3600 / 36000	100.00 ± 0.00%	100.00 ± 0.00%	100.00 ± 0.00% / 96.59 ± 5.97%

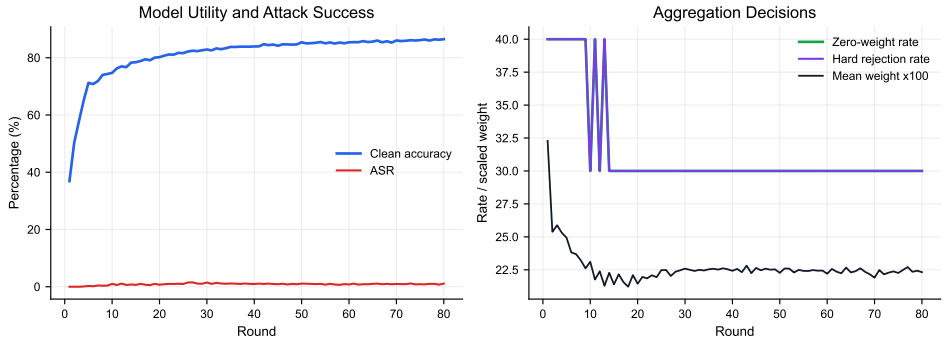

Figure 7: Representative single-run audit trajectory under Fashion-MNIST Non-IID sign-flip attack with the proposed method (seed 42). The figure shows clean accuracy, ASR, zero-weight rate, hard-rejection rate, and mean aggregation weight over 80 rounds. Because this figure visualizes one audit case rather than a multi-seed average, standard-deviation bands are not shown.

6.5. Validator-Backed Finality Under Aggregator Tampering

The preceding audit metrics evaluate whether recorded aggregation decisions can be reconstructed after training. We next evaluate whether a dishonest aggregator can finalize manipulated audit evidence. The validator-backed protocol is tested on the latest 39 formal proposed-method runs, covering 3600 valid audit blocks. For each valid block, we generate invalid proposals by tampering with score fields, aggregation weights, client records, model hashes, previous hashes, client signatures, payload hashes, aggregator authorization, aggregator signatures, and equivocation-style proposal content.

Table 4 summarizes the protocol-security result. Valid proposals finalize in all cases, while all tested invalid proposals are rejected before finality. The invalid-proposal acceptance rate is 0%. This result addresses the core security distinction between a single-server hash chain and validator-backed audit finality: the aggregator can propose a decision, but it cannot unilaterally finalize a manipulated evidence record.

Threshold finality is not unconditional. It depends on the number of validators that are honest and available. Table 5 reports the Byzantine-validator

Figure 8: Per-client aggregation weights in the same representative audit case (seed 42). Malicious clients receive zero aggregation weight across all rounds, while benign clients retain nonzero contribution weights.

Table 4: Validator-backed finality under aggregator and payload tampering. Values summarize 39 proposed-method runs and 3600 valid audit blocks.

Scenario group	Runs	Tampered blocks	Valid finalization	Invalid rejection	Invalid acceptance
Score/weight/payload tampering	39	32400	100.00%	100.00%	0.00%
Aggregator authorization/signature/equivocation tampering	39	10800	100.00%	100.00%	0.00%
All invalid proposals	39	43200	100.00%	100.00%	0.00%

boundary. With five validators and threshold three, invalid proposals are rejected when zero, one, or two validators are Byzantine, but accepted when three Byzantine validators reach the signing threshold. With seven validators and threshold five, the same boundary appears when five Byzantine validators sign invalid proposals. This table is intentionally reported because it makes the trust assumption explicit: the protocol prevents unilateral aggregator manipulation, but it cannot withstand a signing threshold of colluding validators.

Table 6 reports the corresponding liveness boundary under validator dropout. Valid blocks continue to finalize as long as the number of online validators reaches the threshold. Once the available validator count falls below threshold, valid-block finalization fails. This is a liveness failure rather than a poisoning-defense failure, and it clarifies the availability assumption required by the committee design.

Table 5: Byzantine-validator boundary for threshold finality. Values are means over 13 representative runs.

Validators	Threshold	Byzantine validators	Valid finalization	Invalid rejection	Invalid acceptance
5	3	0	100.00%	100.00%	0.00%
5	3	1	100.00%	100.00%	0.00%
5	3	2	100.00%	100.00%	0.00%
5	3	3	100.00%	0.00%	100.00%
7	5	2	100.00%	100.00%	0.00%
7	5	4	100.00%	100.00%	0.00%
7	5	5	100.00%	0.00%	100.00%

Table 6: Validator-dropout liveness boundary. Values are means over 13 representative runs.

Validators	Threshold	Offline validators	Available validators	Valid finalization	Valid failure
5	3	0	5	100.00%	0.00%
5	3	1	4	100.00%	0.00%
5	3	2	3	100.00%	0.00%
5	3	3	2	0.00%	100.00%
7	5	0	7	100.00%	0.00%
7	5	1	6	100.00%	0.00%
7	5	2	5	100.00%	0.00%
7	5	3	4	0.00%	100.00%

6.6. Runtime and Audit Overhead

The proposed method records more per-client information than ordinary robust aggregation baselines, so its audit log is larger. The runtime overhead is moderate in the current implementation. On Fashion-MNIST, the proposed method averages 1.788 s per round, compared with 1.434 s for norm filtering. On CIFAR-10, the proposed method averages 18.719 s per round, close to norm filtering at 18.550 s. The average defense computation time is small relative to total round time. These results indicate that the main cost of the proposed approach is richer audit logging rather than a large increase in aggregation computation.

Validator verification also remains lightweight in the protocol simulation. In the default setting with five validators and threshold three, valid proposal verification takes approximately 1.3–1.6 ms per block depending on the evaluation sweep. Increasing the committee size to seven validators raises verification time to approximately 1.8–2.3 ms per block. These measurements cover local cryptographic checks, commitment verification, proposal consistency checks, and threshold-finality simulation; they do not include real network

Table 7: Runtime and audit-log overhead summary.

Dataset	Method	Runs	Round time (s)	Defense time (s)	Audit log (KB)
Fashion-MNIST	Norm Filter	18	1.434	0.0059	349.28
Fashion-MNIST	Proposed	18	1.788	0.0178	513.59
CIFAR-10	Norm Filter	9	18.550	0.0550	522.76
CIFAR-10	Proposed	9	18.719	0.0797	771.21

propagation, mempool ordering, view-change, or smart-contract execution.

6.7. Implications for Secure Federated Learning

The empirical results support a security-system interpretation rather than a pure accuracy-ranking interpretation. Robustness and auditability are related but distinct goals. Robust aggregation reduces the immediate influence of suspicious updates, while validator-backed auditability preserves and verifies the evidence needed to explain, reproduce, and challenge those decisions after training. A deployment in which operators must investigate incidents, justify client exclusion, or reproduce model-governance records may reasonably prefer a slightly less accurate but auditable defense over a purely filtering-based rule.

This trade-off is relevant to information-security applications where training participants belong to different organizations or administrative domains. In such settings, an aggregation decision can affect future participation, trust, and accountability. Recording the evidence behind down-weighting or exclusion therefore supports not only technical debugging but also operational review and dispute resolution.

The results also show why Non-IID FL complicates security decisions. Honest clients can produce unusual update directions because their local data are not representative of the global distribution. A defense that treats every deviation as malicious risks suppressing useful minority-client information. Conversely, adaptive attackers can exploit this tolerance by making poisoned updates look statistically plausible. The proposed method addresses this tension by combining norm, direction, history, and reputation signals, but the experiments show that this combination should be viewed as a practical trade-off rather than a complete solution.

Finally, the audit overhead results suggest that richer traceability is feasible in the evaluated simulator. The additional defense computation is small relative to training time, especially on CIFAR-10, while the audit log re-

mains modest in absolute size. The validator experiments further clarify the security boundary: threshold finality prevents unilateral aggregator tampering, but it depends on fewer than q validators signing invalid proposals and at least q validators being online for liveness. In a production blockchain or permissioned-ledger deployment, consensus networking, storage replication, view-change, validator selection, and incentive enforcement would add costs beyond those measured here. Therefore, the present results should be interpreted as evidence for the feasibility of validator-backed auditable aggregation logic, not as a full end-to-end blockchain performance evaluation.

7. Limitations and Future Work

The current experiments support the claim that the proposed method is auditable and robustly competitive, but they also reveal important limitations. First, the method is not uniformly clean-accuracy superior to strong filtering baselines. Norm filtering is particularly strong on CIFAR-10 Non-IID, and Trimmed Mean performs best in the Fashion-MNIST malicious-ratio sensitivity experiment. Second, the current benchmark treats FLTrust and FLAME as related-work references rather than empirical baselines. Adding faithful implementations of these methods would strengthen future comparisons. Third, the present validator-backed audit layer is a permissioned threshold-finality simulation rather than a full production blockchain deployment with networking consensus, view change, gas cost, smart-contract execution, or validator incentives. A deployment-focused extension should evaluate those costs separately.

The committee model also has explicit security limits. If a signing threshold of validators colludes or lazily signs invalid proposals, threshold finality cannot prevent invalid block finalization, although the signed votes provide accountability evidence. If too few validators are online, valid blocks cannot finalize. The current design assumes permissioned validator identities and does not implement permissionless Sybil resistance, slashing, stake economics, randomized committee sampling, or validator reputation. These mechanisms are natural extensions but should not be implied by the present simulator.

Future work should focus on adding stronger recent baselines, improving the proposed method's CIFAR-10 clean accuracy without sacrificing auditability, adding DFL-specific collusive poisoning attacks, and extending the

audit layer to a realistic blockchain or permissioned ledger implementation with validator selection, liveness recovery, and incentive enforcement.

8. Conclusion

This paper presents a decentralized validator-backed auditable robust aggregation framework for federated learning under poisoning attacks and aggregator-side tampering. By combining multi-signal anomaly scoring, reputation-aware weighting, client commitments, rotating aggregator proposals, and threshold validator finality, the framework makes aggregation decisions reconstructable while preventing a single aggregator from unilaterally finalizing manipulated audit evidence. The experiments show that the method avoids the catastrophic poisoning failures observed for unprotected FedAvg and maintains low ASR, but the results also show that norm filtering and Trimmed Mean remain strong clean-accuracy baselines. Protocol-level experiments show that invalid proposals are rejected under the validator-threshold assumption and that liveness depends on sufficient validator availability. The contribution should therefore be understood as auditable and competitive robust aggregation with explicit validator-backed security boundaries rather than universal clean-accuracy dominance or a complete production blockchain deployment.

9. Declarations

9.1. Data and Code Availability

The experiments use public image benchmark datasets: MNIST, Fashion-MNIST, and CIFAR-10. The experimental scripts, generated manifests, summary tables, figures, and audit logs are maintained with the project repository. Formal result tables are generated from the consolidated summary CSV using deterministic export scripts, and the audit case study is generated from the recorded per-round and per-client logs. A cleaned reproducibility package containing the paper-specific code, configurations, and analysis scripts is available at `https://github.com/shiqiexperience/hashchain-auditable-robust-fl`.

9.2. Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

9.3. Declaration of Competing Interest

The author declares that there are no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

9.4. Ethics and Security Scope

This study uses public benchmark datasets and does not involve human subjects or private client data. The threat model focuses on poisoning robustness, aggregator-side tampering, validator-backed finality, and reconstructable aggregation decisions in federated learning. Privacy leakage, secure aggregation protocols, permissionless Sybil resistance, full adversarial consensus networking, smart-contract vulnerabilities, and economic incentive attacks are outside the scope of the current implementation and should be evaluated separately in a full deployment study.

9.5. Declaration of Generative AI and AI-Assisted Technologies

During the preparation of this work, the author used OpenAI ChatGPT/Codex to support manuscript organization, language editing, consistency checking, and preparation of submission materials. After using these tools, the author reviewed and edited the content as needed and takes full responsibility for the content of the article.

References

Alqahtani, S., Demirbas, M., 2021. Bottlenecks in blockchain consensus protocols. arXiv preprint arXiv:2103.04234 .

Awan, S., Li, F., Luo, B., Liu, M., 2022. Blockchain for federated learning toward secure distributed machine learning systems: A systemic survey. Soft Computing 26, 4423–4440. doi:10.1007/s00500-021-06496-5.

Bagdasaryan, E., Veit, A., Hua, Y., Estrin, D., Shmatikov, V., 2020. How to backdoor federated learning, in: Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics, PMLR. pp. 2938–2948.

Baruch, G., Baruch, M., Goldberg, Y., 2019. A little is enough: Circumventing defenses for distributed learning, in: Advances in Neural Information Processing Systems, pp. 8632–8642.

Blanchard, P., El Mhamdi, E.M., Guerraoui, R., Stainer, J., 2017. Machine learning with adversaries: Byzantine tolerant gradient descent, in: Advances in Neural Information Processing Systems.

Bonawitz, K., Eichner, H., Grieskamp, W., Huba, D., Ingerman, A., Ivanov, V., Kiddon, C., Konečný, J., Mazzocchi, S., McMahan, H.B., et al., 2019. Towards federated learning at scale: System design, in: Proceedings of Machine Learning and Systems, pp. 374–388.

Bouchiha, M.A., Amara Korba, A., Ghamri-Doudane, Y., 2026. Automated byzantine-resilient clustered decentralized federated learning for battery intelligence in connected EVs. arXiv preprint arXiv:2605.21115 .

Cao, X., Fang, M., Liu, J., Gong, N.Z., 2021. FLTrust: Byzantine-robust federated learning via trust bootstrapping, in: Proceedings of the Network and Distributed System Security Symposium. doi:10.14722/ndss.2021.24434.

Carney, J., Upreti, K., Dagher, G.G., Andersen, T., 2025. FIDELIS: Blockchain-enabled protection against poisoning attacks in federated learning. arXiv preprint arXiv:2508.10042 .

Chen, H., Asif, S.A., Park, J., Shen, C.C., Bennis, M., 2021. Robust blockchained federated learning with model validation and proof-of-stake inspired consensus. arXiv preprint arXiv:2101.03300 .

Dong, N., Wang, Z., Sun, J., Kampffmeyer, M., Knottenbelt, W., Xing, E., 2024. Defending against poisoning attacks in federated learning with blockchain. IEEE Transactions on Artificial Intelligence doi:10.1109/TAI.2024.3376651.

El Mhamdi, E.M., Guerraoui, R., Rouault, S., 2018. The hidden vulnerability of distributed learning in byzantium, in: Proceedings of the 35th International Conference on Machine Learning, PMLR. pp. 3521–3530.

Fang, M., Cao, X., Jia, J., Gong, N.Z., 2020. Local model poisoning attacks to byzantine-robust federated learning, in: Proceedings of the 29th USENIX Security Symposium, USENIX Association. pp. 1605–1622.

Fung, C., Yoon, C.J.M., Beschastnikh, I., 2020. The limitations of federated learning in sybil settings, in: Proceedings of the 23rd International Symposium on Research in Attacks, Intrusions and Defenses, pp. 301–316.

García-Márquez, M., Rodríguez-Barroso, N., Luzón, M.V., Herrera, F., 2025. Krum federated chain (KFC): Using blockchain to defend against adversarial attacks in federated learning. arXiv preprint arXiv:2502.06917 .

Hallaji, E., Razavi-Far, R., Saif, M., Wang, B., Yang, Q., 2024. Decentralized federated learning: A survey on security and privacy. arXiv preprint arXiv:2401.17319 .

Jiang, Y., Ma, B., Wang, X., Yu, P., Yu, G., Wang, Z., Ni, W., Liu, R.P., 2023. Blockchained federated learning for internet of things: A comprehensive survey. arXiv preprint arXiv:2305.04513 .

Kairouz, P., McMahan, H.B., Avent, B., Bellet, A., Bennis, M., Bhagoji, A.N., Bonawitz, K., Charles, Z., Cormode, G., Cummings, R., et al., 2021. Advances and open problems in federated learning. Foundations and Trends in Machine Learning 14, 1–210. doi:10.1561/2200000083.

Karimireddy, S.P., Kale, S., Mohri, M., Reddi, S., Stich, S.U., Suresh, A.T., 2020. SCAFFOLD: Stochastic controlled averaging for federated learning, in: Proceedings of the 37th International Conference on Machine Learning, PMLR. pp. 5132–5143.

Li, J., Shao, Y., Wei, K., Ding, M., Ma, C., Shi, L., Han, Z., Poor, H.V., 2021. Blockchain assisted decentralized federated learning (BLADE-FL): Performance analysis and resource allocation. arXiv preprint arXiv:2101.06905 .

Li, T., Sahu, A.K., Zaheer, M., Sanjabi, M., Talwalkar, A., Smith, V., 2020a. Federated optimization in heterogeneous networks, in: Proceedings of Machine Learning and Systems, pp. 429–450.

Li, Y., Chen, C., Liu, N., Huang, H., Zheng, Z., Yan, Q., 2020b. A blockchain-based decentralized federated learning framework with committee consensus. arXiv preprint arXiv:2004.00773 .

Lu, Y., Huang, X., Dai, Y., Maharjan, S., Zhang, Y., 2020. Blockchain and federated learning for privacy-preserved data sharing in industrial IoT.

IEEE Transactions on Industrial Informatics 16, 4177–4186. doi:10.1109/TII.2019.2942190.

McMahan, B., Moore, E., Ramage, D., Hampson, S., Agüera y Arcas, B., 2017. Communication-efficient learning of deep networks from decentralized data, in: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics, PMLR. pp. 1273–1282.

Mothukuri, V., Parizi, R.M., Pouriyeh, S., Huang, Y., Dehghantanha, A., Srivastava, G., 2021. A survey on security and privacy of federated learning. Future Generation Computer Systems 115, 619–640. doi:10.1016/j.future.2020.10.007.

National Institute of Standards and Technology, 2024. Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations. Technical Report NIST AI 100-2e2023. National Institute of Standards and Technology. doi:10.6028/NIST.AI.100-2e2023.

Nguyen, D.C., Ding, M., Pham, Q.V., Pathirana, P.N., Le, L.B., Seneviratne, A., Li, J., Niyato, D., Poor, H.V., 2021. Federated learning meets blockchain in edge computing: Opportunities and challenges. IEEE Internet of Things Journal 8, 12806–12825. doi:10.1109/JIOT.2021.3072611.

Nguyen, T.D., Rieger, P., Chen, H., Yalame, H., Möllering, H., Fereidooni, H., Marchal, S., Miettinen, M., Mirhoseini, A., Zeitouni, S., Koushanfar, F., Sadeghi, A.R., Schneider, T., 2022. FLAME: Taming backdoors in federated learning, in: Proceedings of the 31st USENIX Security Symposium, USENIX Association.

Pillutla, K., Kakade, S.M., Harchaoui, Z., 2022. Robust aggregation for federated learning. IEEE Transactions on Signal Processing 70, 1142–1154. doi:10.1109/TSP.2022.3153135.

Qin, Z., Yan, X., Zhou, M., Deng, S., 2022. BlockDFL: A blockchain-based fully decentralized peer-to-peer federated learning framework. arXiv preprint arXiv:2205.10568 .

Rajab, T., Manshaei, M.H., Dakhilalian, M., Jadliwala, M., Rahman, M.A., 2020. On the feasibility of sybil attacks in shard-based permissionless blockchains. arXiv preprint arXiv:2002.06531 .

Reddi, S.J., Charles, Z., Zaheer, M., Garrett, Z., Rush, K., Konečný, J., Kumar, S., McMahan, H.B., 2021. Adaptive federated optimization, in: International Conference on Learning Representations.

Ren, Y.L., Hu, M., Yang, Z., Feng, G., Zhang, X., 2024. BPFL: Blockchain-based privacy-preserving federated learning against poisoning attack. Information Sciences 665, 120377. doi:10.1016/j.ins.2024.120377.

Shayan, M., Fung, C., Yoon, C.J.M., Beschastnikh, I., 2018. Biscotti: A ledger for private and secure peer-to-peer machine learning. arXiv preprint arXiv:1811.09904 .

Wang, X., Ye, B., Xu, L., Wu, L., Hsieh, S.Y., Wu, J., Lin, L., 2025a. FedHAN: A cache-based semi-asynchronous federated learning framework defending against poisoning attacks in heterogeneous clients, in: Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence, pp. 3407–3416. doi:10.24963/ijcai.2025/379.

Wang, Y., Zhang, H., Wen, S., Qiu, W., Guo, B., 2025b. Defending against sophisticated poisoning attacks with RL-based aggregation in federated learning. Proceedings of the AAAI Conference on Artificial Intelligence 39, 25443–25451. doi:10.1609/aaai.v39i24.34733.

Xie, C., Huang, K., Chen, P.Y., Li, B., 2020. DBA: Distributed backdoor attacks against federated learning, in: International Conference on Learning Representations.

Yin, D., Chen, Y., Kannan, R., Bartlett, P., 2018. Byzantine-robust distributed learning: Towards optimal statistical rates, in: Proceedings of the 35th International Conference on Machine Learning, PMLR. pp. 5650–5659.

Yuan, L., Wang, Z., Sun, L., Yu, P.S., Brinton, C.G., 2024. Decentralized federated learning: A survey and perspective. arXiv preprint arXiv:2306.01603 .

Zhang, G., Pan, F., Mao, Y., Tijanic, S., Dang'ana, M., Motepalli, S., Zhang, S., Jacobsen, H.A., 2022. Reaching consensus in the byzantine empire: A comprehensive review of BFT consensus algorithms. arXiv preprint arXiv:2204.03181 .

Zhang, T., Ying, C., Xia, F., Jin, H., Luo, Y., Wei, D., Yu, X., Xu, Y., Jiang, X., Zhang, W., Tao, D., 2024. BIT-FL: Blockchain-enabled incentivized and secure federated learning framework. IEEE Transactions on Mobile Computing doi:10.1109/TMC.2024.3477616.